

Introduction to DOM and JavaScript

The Document Object Model (DOM) is a programming interface that represents the structure of a web page as a tree of objects. With the DOM, JavaScript can access, manipulate, and update the content, structure, and styles of a webpage dynamically.

Why is the DOM important?

- It allows you to change text, images, and styles without reloading the page.
- It enables you to make webpages interactive by responding to user actions (clicks, typing, hovering, etc.).
- It forms the foundation for building modern web applications.

In this guide, you will learn how to:

- Select elements using different DOM methods.
- Edit and insert content into a webpage.
- Handle events to make webpages interactive.
- Work with parent/child nodes and classes.

Build small projects like a calculator, counter app, and to-do list using DOM manipulation.

2. Selecting Elements

(i) Using `getElementById()`: Selects an element by its id:

```
<body>
  <h1 id="title">Hello, Javascript</h1>
  <script>
    const heading = document.getElementById("title");
    console.log(heading); // Output: Hello, Javascript
  </script>
</body>
```

Explanation

1. `document.getElementById("title")`:

- This method searches the HTML document for an element with the id of "title".
- It assigns this `<h1>` element to the variable **heading**.

2. `heading.innerText`:

`.innerText` retrieves the text content inside the `<h1>` element.

Here, the text is "Hello, Javascript".

3. `console.log(heading.innerText);`:

This prints the text "Hello, Javascript" to the browser's console.

(ii) The `getElementsByClassName()` method: returns a collection of elements with a specified class name(s).

Example

```
<body>
  <p class="text">This is a paragraph</p>
  <p class="text">This is another paragraph</p>
  <script>

    let parags = document.getElementsByClassName('text');
    console.log(parags); // HTMLCollection(2)

    //to cycle through all the items in the collection
    for( i = 0; i < parags.length; i++){
      console.log(parags[i]);
    }

    //to use forEach, convert the collection to Array
    Array.from(parags).forEach(function(parag) {
      console.log(parag);
    })
  </script>
</body>
```

(iii) The `getElementsByTagName()` method : returns a collection of all elements with a specified tag name.

Example:

```
<body>
  <ul>
    <li>Oranges</li>
    <li>pineapple</li>
    <li>Banana</li>
  </ul>
  <script>
    list_of_fruits = document.getElementsByTagName('li');
    console.log(list_of_fruits) //Output: HTMLCollection(3)

    console.log(list_of_fruits[0]); //Output: <li>Oranges</li>

    //to cycle through all items in the collection
    for(i = 0; i < list_of_fruits.length; i++){
      console.log(list_of_fruits[i]);
    }

    //another method is to use forEach
    Array.from(list_of_fruits).forEach(function(fruit) {
      console.log(fruit);
    })
  </script>
</body>
```

```
    })  
  </script>  
</body>
```

(ii) Using `querySelector()`: Selects the first matching element:

It allows you to select elements using any valid CSS selector (classes, IDs, attributes, tags, etc.), making it very flexible.

Example:

```
<body>  
  <p>This is a paragraph</p>  
  <p>This is another paragraph</p>  
  
  <script>  
    const paragraph = document.querySelector("p");  
    console.log(paragraph.innerText); //Output: <p>This is a paragraph</p>  
  </script>  
</body>
```

Using `querySelectorAll()`

Selects all matching elements (returns a `NodeList`):

NB: Because it returns a `nodeList`, `forEach` method can be used on it without the need to convert it into an `Array`.

Example:

```
<body>  
  <h2>Today's Tasks:</h2>  
  <ul>  
    <li class="item">Buy groceries</li>  
    <li class="item">Complete JavaScript tutorial</li>  
    <li class="item">Call the plumber</li>  
  </ul>  
  
  <script>  
    const items = document.querySelectorAll(".item");  
    items.forEach(item => console.log(item));  
  </script>  
</body>
```

3. Editing HTML and Inserting Text into the DOM

You can change the content, styles, and attributes of elements dynamically.

Changing Content

(i) **.textContent**: The textContent property sets or returns the text content of the specified node, and all its descendants.

For example:

```
<body>
  <p id="first_paragraph">This is a paragraph</p>
  <p id="second_paragraph">This is another paragraph</p>

  <script>
    const parag = document.getElementById('first_paragraph');
    const updated_parag = parag.textContent = 'This is a first paragraph';
    console.log(updated_parag); //Output: This is a paragraph
  </script>
</body>
```

(ii) **.innerText**: Return the text content of an element or node

```
<body>
  <p>this is the first paragraph</p>
  <p>this is the second paragraph</p>
  <script>
    const first_paragraph = document.querySelector('p').innerText = "This is
my first paragraph";
    console.log(first_paragraph); //Output: this is a paragraph
  </script>
</body>
```

EVENT

Events are used to make web pages interactive. They allow you to perform actions when a user interacts with the page, like:

- Showing a message when a button is clicked
- Changing the color of an element when the mouse hovers over it
- Validating a form before submission

Commonly Used Events:

1. click – When an element is clicked
2. mouseover – When the mouse pointer is over an element
3. mouseout – When the mouse pointer leaves an element
4. keydown – When a key is pressed on the keyboard
5. load – When the webpage finishes loading
6. submit – When a form is submitted

How to Use Events:

There are three main ways to add events in JavaScript:

1. Inline Event Handler (in HTML)
2. Using the DOM's Event Property (in JavaScript)
3. Using `addEventListener()` (Recommended)

1. Inline Event Handler (Not Recommended):

```
<button onclick="alert('I was clicked')">Click me</button>
```

2. Using `addEventListener()` (Best Practice):

```
<body>
  <button onclick="alert('I was clicked')">Click me</button>

  <button id="myButton">Click me</button>

  <script>
    const mybutton = document.getElementById('myButton');
    mybutton.addEventListener('click', function(){
      alert('Button clicked using addEventListener!');
    })
  </script>

</body>
```

Example: Change Text on Button Click

This example changes the text inside the `<p>` tag when the button is clicked. It uses `textContent` to update the text.

```
<body>

  <P id="text">Hello,</P>
  <button id="changeText">Change Text</button>

  <script>
    const text = document.getElementById('text');
    const mybutton2 = document.getElementById('changeText');
    mybutton2.addEventListener('click', function(){
      text.textContent = 'You clicked the button!';
    })
  </script>

</body>
```

Sample Project 1: Addition of Two Numbers

```
<body>
  <label for="num1">Enter First Number:</label>
  <input type="number" name="num1" id="num1" placeholder="Enter Number 1">
  <label for="num2">Enter Second Number: </label>
  <input type="number" name="num2" id="num2" placeholder="Enter Number 2">
  <button id="addButton">Add</button>
  <p id="result"></p>

  <script>
    // Get elements by their IDs
    const num1 = document.getElementById('num1');
    const num2 = document.getElementById('num2');
    const addButton = document.getElementById('addButton');
    const result = document.getElementById('result');

    // Add event listener to the button
    addButton.addEventListener('click', function(){
      // Get the values from the input fields and convert them to numbers
      const value1 = Number(num1.value);
      const value2 = Number(num2.value);

      // Add the two numbers and display the result
      result.textContent = value1 + value2;
    })
  </script>
</body>
```

The CSS code

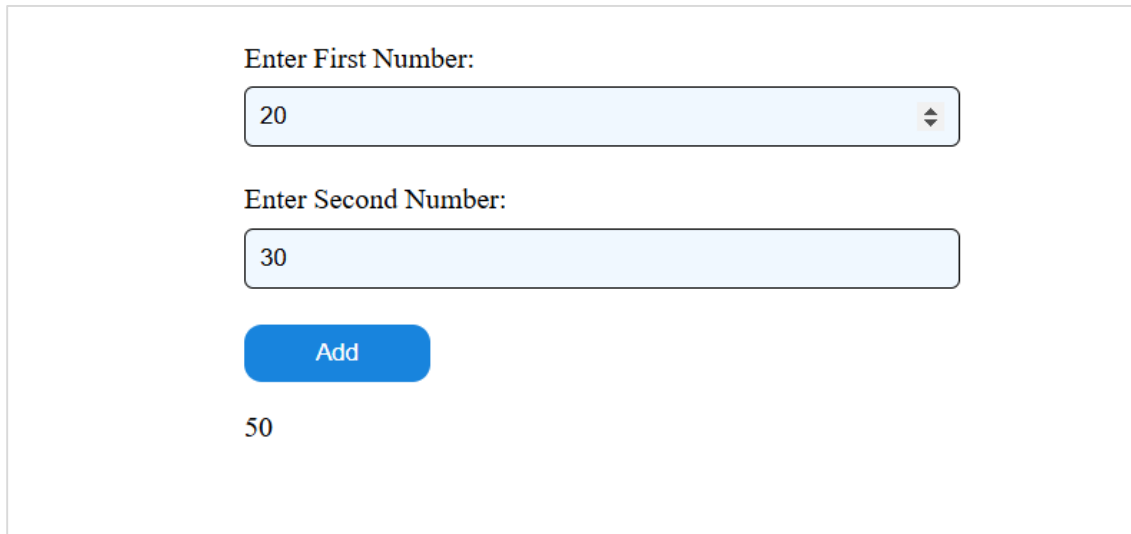
```
<style>
  body{
    width: 400px;
    margin: 30px auto;
  }
  input{
    width: 400px;
    padding: 8px;
    margin-bottom: 20px;
    margin-top: 7px;
    border-radius: 5px;
    border: 1px solid black;
    background-color: aliceblue;
  }
```

```

button{
  padding: 8px 40px;
  background-color: rgb(24, 132, 221);
  color: white;
  border-radius: 10px;
  border: none;
  cursor: pointer;
}
</style>

```

Output



The screenshot shows a web form with the following elements:

- A label "Enter First Number:" above a light blue input field containing the value "20".
- A label "Enter Second Number:" above a light blue input field containing the value "30".
- A blue button with the text "Add" below the input fields.
- The number "50" displayed below the button, representing the sum of the two numbers.

Sample Project 2: Simple Calculator

```

<div class="container">
  <h2>Simple Calculator</h2>
  <input type="number" id="num1" placeholder="Enter first number">

  <!-- Dropdown for choosing operation -->
  <select id="operation">
    <option value="+">+</option>
    <option value="-">-</option>
    <option value="*">*</option>
    <option value="/">/</option>
  </select>

  <input type="number" id="num2" placeholder="Enter second number">
  <button id="calculateButton">Calculate</button>
  <button id="clearButton">Clear</button>

```

```
<p>Result: <span id="result"></span></p>
</div>
```

```
<script>
    // Get elements by their IDs
    const num1 = document.getElementById('num1');
    const num2 = document.getElementById('num2');
    const operation = document.getElementById('operation');
    const calculateButton = document.getElementById('calculateButton');
    const clearButton = document.getElementById('clearButton');
    const result = document.getElementById('result');

    // Clear the input fields
    clearButton.addEventListener('click', function(){
        num1.value = '';
        num2.value = '';
    })

    // Add event listener to the button
    calculateButton.addEventListener('click', function() {
        // Get the values from the input fields and convert them to numbers
        const value1 = Number(num1.value);
        const value2 = Number(num2.value);
        const op = operation.value;
        let calculation;

        // Perform the selected operation
        if (op === '+') {
            calculation = value1 + value2;
        } else if (op === '-') {
            calculation = value1 - value2;
        } else if (op === '*') {
            calculation = value1 * value2;
        } else if (op === '/') {
            calculation = value1 / value2;
        }

        // Display the result
        result.textContent = calculation;

        // Clear the input fields after calculation
        clearFields();
    });
</script>
```



```
</script>

<style>
  body{
    display: flex;
    flex-direction: row;
  }
  .container{
    width: 400px;
    margin: 50px auto;
  }

  input{
    width: 400px;
    margin-bottom: 20px;
    padding: 5px;
  }
  select{
    width: 413px;
    margin-bottom: 20px;
    padding: 5px;
  }
  button{
    padding: 5px 20px;
    color: white;
    background-color: rgb(36, 109, 192);
    border-radius: 5px;
    border: none;
  }
</style>
```

Simple Calculator

+

▼

Calculate

Clear

Result: 50

Sample Project 3: Counter App

<body>

```
<h1>Counter App</h1>
<p id="counter">0</p>
<button id="increment">Increment</button>
<button id="decrement">Decrement</button>
<button id="reset">Reset</button>

<script>
  // Get the counter element and buttons
  const counter = document.getElementById("counter");
  const incrementBtn = document.getElementById("increment");
  const decrementBtn = document.getElementById("decrement");
  const resetBtn = document.getElementById("reset");

  let count = 0; // Initial counter value

  // Function to update counter display
  function updateCounter() {
    counter.innerText = count;
  }

  // Increment button click event
  incrementBtn.addEventListener("click", () => {
    count++;
    updateCounter();
  });

  // Decrement button click event
  decrementBtn.addEventListener("click", () => {
    if (count > 0) {
      count--;
      updateCounter();
    }
  });

  // Reset button click event
  resetBtn.addEventListener("click", () => {
    count = 0;
    updateCounter();
  });

  // Initial display update
```

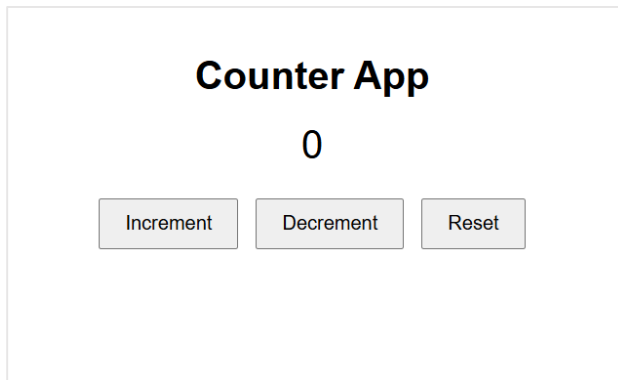
```
        updateCounter();
    </script>

</body>
```

Internal CSS

```
<style>
    body {
        font-family: Arial, sans-serif;
        text-align: center;
        margin-top: 50px;
    }
    #counter {
        font-size: 2rem;
        margin: 20px;
    }
    button {
        font-size: 1rem;
        padding: 10px 20px;
        margin: 5px;
        cursor: pointer;
    }
</style>
```

Output



PARENT/CHILD NODE IN THE DOM TREE STRUCTURE

The **DOM Tree** represents an HTML document in a hierarchical format:

- Elements are nodes in the tree.
- A parent node contains one or more child nodes.
- Nodes can also have siblings (elements at the same level).

Example: Simple HTML Structure

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="container">
    <p>This is paragraph</p>
```

```
        <button>Click Me</button>
    </div>
</body>
</html>
```

html

```
├─ head
├─ body
│   └─ div#container (Parent)
│       ├── p (Child)
│       └─ button (Child)
```

Traversing the DOM (Parent-Child Navigation)

- (i) Accessing a Parent Element
 - `parentElement` – Gets the parent of an element.

```
<script>
    let paragraph = document.querySelector('p');
    console.log(paragraph.parentElement); // Logs <div id="container">
</script>
```

Accessing Child Elements

- (i) `children` – Returns all child elements of a parent.

```
<script>
    let container = document.getElementById('container');
    console.log(container.children); // Logs a list of child elements
</script>
```

- (ii) `firstElementChild` / `lastElementChild` – Get first/last child.

```
<script>
    console.log(container.firstElementChild); // Logs <p>
    console.log(container.lastElementChild); // Logs <button>
</script>
```

Accessing Sibling Elements

nextElementSibling – Moves to the next sibling.

previousElementSibling – Moves to the previous sibling.

```
<script>
    let para = document.querySelector('p');
    console.log(para.nextElementSibling); // Logs <button>
</script>
```

Adding/Removing Parent & Child Elements

Adding a New Child Element

We can dynamically add new child elements using `appendChild()`:

```
let newParagraph = document.createElement('p');
newParagraph.innerText = "This is a new Paragraph";
document.getElementById('container').appendChild(newParagraph);
```

This creates a new `<p>` element and adds it as a child of `#container`.

Removing a Child Element

We can remove a specific child using `removeChild()`:

```
let parentDiv = document.getElementById('container');
let firstChild = parentDiv.firstChild;
parentDiv.removeChild(firstChild); // Removes <p>
```

Practical Example: Basic To-Do List Using Parent-Child Relationship

HTML

```
<div id="taskList">
    <h3>Tasks</h3>
    <ul id="tasks"></ul>
    <button id="addTask">Add Task</button>
</div>
```

```
document.getElementById("addTask").addEventListener("click", function() {
    let task = document.createElement("li"); // Create <li>
    task.innerText = "New Task";
    document.getElementById("tasks").appendChild(task); // Append as child
```

Tasks

- New Task
- New Task
- New Task
- New Task

Add Task

CLASSLIST

In JavaScript, the `classList` property provides methods to manipulate an element's classes dynamically.

Let's break down each method with explanations and examples.

1. `classList.add()` – Adding a Class

The `add()` method adds a specified class to an element. If the class is already present, it won't be added again.

Example:

```
<style>
    .active {
        background-color: green;
        color: white;
        padding: 10px;
    }
</style>
```

```

<body>
  <button id="activateBtn">Activate</button>
  <div id="box">I am a box</div>

  <script>
    let activateBtn = document.getElementById('activateBtn');
    let box = document.getElementById('box');
    activateBtn.addEventListener('click', function(){
      box.classList.add('active');
    })
  </script>
</body>

```

What Happens?

When you click the "Activate" button, the box turns green because the "active" class is added.

2. `classList.remove()` – Removing a Class

Example

```

<body>
  <button id="deactivateBtn">Deactivate</button>
  <div id="box" class="active">I am active</div>

  <script>

    let deactivateBtn = document.getElementById('deactivateBtn');
    box = document.getElementById('box');
    deactivateBtn.addEventListener('click', function(){
      box.classList.remove('active');
    })
  </script>
</body>

```

3. `classList.toggle()` – Adding & Removing a Class Dynamically

The `toggle()` method adds the class if it's missing and removes it if it's already present.

Example: Dark Mode Toggle

```

<body>

  <button id="toggleTheme">Toggle Dark Mode</button>

```



```

<div id="content">Switch between light and dark mode</div>

<script>

    let toggleTheme = document.getElementById('toggleTheme');
    let content = document.getElementById('content');
    toggleTheme.addEventListener('click', function(){
        content.classList.toggle('dark-mode');
    })
</script>
</body>

.dark-mode {
    background-color: black;
    color: white;
    padding: 10px;
}

```

What Happens?

Clicking the button switches between dark mode (black background) and light mode (default).

4. `classList.contains()` - Checking If an Element Has a Class

```

<body>

    <button id="checkBtn">Check Active</button>
    <div id="box" class="active">Is this active?</div>

    <script>

        let checkBtn = document.getElementById('checkBtn');
        let box = document.getElementById('box');
        checkBtn.addEventListener('click', function(){
            if(box.classList.contains('active')){
                alert('box is active');
            } else {
                alert('box is not active');
            }
        })
    </script>
</body>

```

What Happens?

Clicking "Check Active" shows an alert stating whether "active" is present.

PROJECT SAMPLE: SIMPLE TODO-LIST APPLICATION

HTML AND JAVASCRIPT

```
<body>
  <div class="container">
    <h2>To-Do List</h2>
    <input type="text" id="taskInput" placeholder="Enter task">
    <button id="addTask">Add</button>
    <ul id="taskList"></ul>
  </div>
  <script>
    const taskInput = document.getElementById("taskInput");
    const addTaskBtn = document.getElementById("addTask");
    const taskList = document.getElementById("taskList");

    addTaskBtn.addEventListener("click", function () {
      const taskText = taskInput.value.trim();
      if (taskText === "") return; // Ignore empty input
```

```

    const li = document.createElement("li");
    li.innerText = taskText;

    // Toggle task completion
    li.addEventListener("click", function () {
        li.classList.toggle("completed");
    });

    // Remove task
    const removeBtn = document.createElement("button");
    removeBtn.innerText = "Delete";
    removeBtn.addEventListener("click", function(){
        li.remove();
    });

    li.appendChild(removeBtn);
    taskList.appendChild(li);
    taskInput.value = ""; // Clear input
  });
</script>

```

</body>

CSS

<style>

```

  body{
    font-family:'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  }
  /* Task Completion Style */
  .completed {
    text-decoration: line-through;
    color: gray;
  }
  .container{
    width: 350px;
    margin: 40px auto;
  }
  input{
    width: 250px;
    padding: 5px 10px;
  }
  button{
    padding: 7px 20px;
    color: white;
    background-color: rgb(25, 97, 198);
  }

```

```

        border: none;
        border-radius: 5px;
    }

    li{
        display: flex;
        justify-content: space-between;
        width: 350px;
    }

    li button{
        background: none;
        color: red;
    }
    ul{
        list-style-type: none;
        padding-left: 0;
    }
</style>

```

OUTPUT

To-Do List

tidy up the houseDelete

cook the mealDelete

go to the shopping mallDelete

Study for my examsDelete